R final assignment

Tang Jinchi 25204395

Part 1 Analysis

This section analyses Ireland's climate over the past 60 years using four datasets (wind.csv, sunshine.csv, temperature.csv, and meteorological.csv), which represent four distinct domains of climatic data. The purpose of this analysis is to identify general climatic patterns and relationships among key variables.

1.1 preprocess the dataset

First we need to import relevant packages and datasets.

1 Import pacakges

```
library(dplyr)
library(tibble)
library(lubridate)
library(purrr)
Sys.setlocale("LC_TIME", "C")
```

```
[1] "C"
```

2 Import dataset

```r
wind <- read.csv("wind.csv", header = TRUE, stringsAsFactors = FALSE)
sunshine <- read.csv("sunshine.csv", header = TRUE, stringsAsFactors = FALSE)
temperature <- read.csv("temperature.csv", header = TRUE, stringsAsFactors = FALSE)
meteorological <- read.csv("meteorological.csv", header = TRUE, stringsAsFactors = FALSE)


wind <- tibble::as_tibble(wind)
sunshine <- tibble::as_tibble(sunshine)
temperature <- tibble::as_tibble(temperature)
meteorological <- tibble::as_tibble(meteorological)



datasets <- list(
  wind = wind,
  sunshine = sunshine,
  temperature = temperature,
  meteorological = meteorological
)

map(datasets, dim)
```

```
$wind
[1] 18960     5

$sunshine
[1] 9480    5

$temperature
[1] 56880     5

$meteorological
[1] 9480    5
```

The purrr::map() function is used to apply the same operation across multiple datasets, allowing a concise and consistent comparison of their dimensions.The result shows that the observations among the 4 datasets are different from one another.So the next step is to explore the structure of the four datasets.
The temperature dataset has the largest number of observations, containing about six times more records than the wind and meteorological datasets. Therefore, the next section explores the relationships among the datasets and examines possible approaches for merging the data frames.

3 Explore the relationship among data frames

First we can try to find whether all of the four datasets have the same stations.

```
station_counts <- list(
wind = length(unique(wind$`Meteorological.Weather.Station`)),
sunshine = length(unique(sunshine$`Meteorological.Weather.Station`)),
temperature = length(unique(temperature$`Meteorological.Weather.Station`)),
meteorological = length(unique(meteorological$`Meteorological.Weather.Station`))
)

station_counts
```

```
$wind
[1] 12

$sunshine
[1] 12

$temperature
[1] 12

$meteorological
[1] 12
```

The four datasets contain the same number of meteorological stations. Therefore, the next step is to examine differences in the labels across datasets.

```
all_statistics <- data.frame(
dataset = c(
rep("temperature", length(unique(temperature$Statistic.Label))),
rep("wind", length(unique(wind$Statistic.Label))),
rep("sunshine", length(unique(sunshine$Statistic.Label))),
rep("meteorological", length(unique(meteorological$Statistic.Label)))
),
statistic_label = c(
sort(unique(temperature$Statistic.Label)),
sort(unique(wind$Statistic.Label)),
sort(unique(sunshine$Statistic.Label)),
sort(unique(meteorological$Statistic.Label))
)
```

```
)

all_statistics
```

```
          dataset           statistic_label
1     temperature Grass Minimum Temperature
2     temperature   Maximum Air Temperature
3     temperature      Mean Air Temperature
4     temperature  Mean Maximum Temperature
5     temperature  Mean Minimum Temperature
6     temperature   Minimum Air Temperature
7            wind              Highest Gust
8            wind           Mean Wind Speed
9         sunshine         Sunshine Duration
10 meteorological      Precipitation Amount
```

The temperature dataset records six different statistical measures per station and month, whereas the wind dataset records only two. Sunshine and meteorological variables contain a single measure. This difference in measurement granularity explains the large disparity in the number of observations across datasets.In order to obtain the final dataset we need,we should first divide the temperature and wind dataset and then combine them together.

4 Divide data frames

The filter() function is applied to split data frames by statistical label. As the sunshine and precipitation datasets each contain only a single label, they are not further divided but are instead renamed for clarity.

```
temp_mean_air <- temperature |>
filter(Statistic.Label == "Mean Air Temperature")

temp_mean_max <- temperature |>
filter(Statistic.Label == "Mean Maximum Temperature")

temp_mean_min <- temperature |>
filter(Statistic.Label == "Mean Minimum Temperature")

temp_max_air <- temperature |>
filter(Statistic.Label == "Maximum Air Temperature")
```

```r
temp_min_air <- temperature |>
filter(Statistic.Label == "Minimum Air Temperature")

temp_grass_min <- temperature |>
filter(Statistic.Label == "Grass Minimum Temperature")

wind_mean <- wind |>
filter(Statistic.Label == "Mean Wind Speed")

wind_gust <- wind |>
filter(Statistic.Label == "Highest Gust")

sunshine_duration <- sunshine
precipitation <- meteorological
```

5 Rename the columns

Since all of the statistic values share the same name "VALUE", they need to be renamed for clarity after merging.

```r
# --- Temperature (all 6 labels) ---

temp_mean_air <- temp_mean_air |>
select(Meteorological.Weather.Station, Month, VALUE) |>
rename(mean_air_temperature = VALUE)

temp_mean_max <- temp_mean_max |>
select(Meteorological.Weather.Station, Month, VALUE) |>
rename(mean_max_temperature = VALUE)

temp_mean_min <- temp_mean_min |>
select(Meteorological.Weather.Station, Month, VALUE) |>
rename(mean_min_temperature = VALUE)

temp_max_air <- temp_max_air |>
select(Meteorological.Weather.Station, Month, VALUE) |>
rename(max_air_temperature = VALUE)

temp_min_air <- temp_min_air |>
select(Meteorological.Weather.Station, Month, VALUE) |>
rename(min_air_temperature = VALUE)
```

```r
temp_grass_min <- temp_grass_min |>
select(Meteorological.Weather.Station, Month, VALUE) |>
rename(grass_min_temperature = VALUE)

# --- Wind ---

wind_mean <- wind_mean |>
select(Meteorological.Weather.Station, Month, VALUE) |>
rename(mean_wind_speed = VALUE)

wind_gust <- wind_gust |>
select(Meteorological.Weather.Station, Month, VALUE) |>
rename(highest_gust = VALUE)

# --- Sunshine & Precipitation ---

sunshine_duration <- sunshine_duration |>
select(Meteorological.Weather.Station, Month, VALUE) |>
rename(sunshine_hours = VALUE)

precipitation <- precipitation |>
select(Meteorological.Weather.Station, Month, VALUE) |>
rename(precipitation_mm = VALUE)
```

6 Merge the dataframes

Because all of the data frames share the same values in the two columns "Meteorological.Weather.Station" and "Month", these two columns can be used as the keys for a left join.

```r
weather_wide <- temp_mean_air |>
left_join(temp_mean_max, by = c("Meteorological.Weather.Station", "Month")) |>
left_join(temp_mean_min, by = c("Meteorological.Weather.Station", "Month")) |>
left_join(temp_max_air, by = c("Meteorological.Weather.Station", "Month")) |>
left_join(temp_min_air, by = c("Meteorological.Weather.Station", "Month")) |>
left_join(temp_grass_min, by = c("Meteorological.Weather.Station", "Month")) |>
left_join(wind_mean, by = c("Meteorological.Weather.Station", "Month")) |>
left_join(wind_gust, by = c("Meteorological.Weather.Station", "Month")) |>
left_join(sunshine_duration, by = c("Meteorological.Weather.Station", "Month")) |>
left_join(precipitation, by = c("Meteorological.Weather.Station", "Month"))
```

Finally, we can check the resulting data frame by examining its dimensions and column names.

```
head(weather_wide)
```

```
# A tibble: 6 x 12
  Meteorological.Weather.Station Month mean_air_temperature mean_max_temperature
  <chr>                          <chr>                <dbl>                <dbl>
1 Malin Head                     1960~                  5.7                  8
2 Claremorris                    1960~                  3.8                  7
3 Valentia Observatory           1960~                  6.6                  9.5
4 Belmullet                      1960~                  5.6                  8.3
5 Shannon Airport                1960~                  4.7                  8.2
6 Dublin Airport                 1960~                  5.1                  7.7
# i 8 more variables: mean_min_temperature <dbl>, max_air_temperature <dbl>,
#   min_air_temperature <dbl>, grass_min_temperature <dbl>,
#   mean_wind_speed <dbl>, highest_gust <dbl>, sunshine_hours <dbl>,
#   precipitation_mm <dbl>
```

```
dim(weather_wide)
```

```
[1] 9480   12
```

7 Change the format of date

```
weather_wide$month_date <- as.Date(
paste0("01 ", weather_wide$Month),
format = "%d %Y %B"
)

weather_wide$Year <- as.integer(format(weather_wide$month_date, "%Y"))
weather_wide$Month <- as.integer(format(weather_wide$month_date, "%m"))


weather_wide <- weather_wide |>
select(-month_date,)

head(weather_wide)
```

```
# A tibble: 6 x 13
  Meteorological.Weather.Station Month mean_air_temperature mean_max_temperature
  <chr>                           <int>                <dbl>                <dbl>
1 Malin Head                          1                  5.7                  8
2 Claremorris                         1                  3.8                  7
3 Valentia Observatory                1                  6.6                  9.5
4 Belmullet                           1                  5.6                  8.3
5 Shannon Airport                     1                  4.7                  8.2
6 Dublin Airport                      1                  5.1                  7.7
# i 9 more variables: mean_min_temperature <dbl>, max_air_temperature <dbl>,
#   min_air_temperature <dbl>, grass_min_temperature <dbl>,
#   mean_wind_speed <dbl>, highest_gust <dbl>, sunshine_hours <dbl>,
#   precipitation_mm <dbl>, Year <int>
```

8 Check N/A Value

The next section focuses on identifying missing (N/A) values and determining an appropriate approach for handling them.By viewing the data, we can find that stations like Cork airport and Casement have a lot of N/A values in the early years in 1960s.We can hence infer that during some early years, there are several stations were not built.So We need to group the data by station and year in order to find these stations.

```r
#gather the data by the year and stations

na_by_station_year <- weather_wide |>
group_by(Meteorological.Weather.Station, Year) |>
summarise(
na_count = sum(is.na(across(where(is.numeric)))),
.groups = "drop"
)


na_year_range <- na_by_station_year |>
filter(na_count == 120) |>
group_by(Meteorological.Weather.Station) |>
summarise(
first_na_year = min(Year),
last_na_year = max(Year),
.groups = "drop"
)

na_year_range
```

```
# A tibble: 5 x 3
  Meteorological.Weather.Station first_na_year last_na_year
  <chr>                                 <int>        <int>
1 Casement                               1960         1963
2 Cork Airport                           1960         1961
3 Markree                                1960         2004
4 Phoenix Park                           1960         2003
5 Roches Point                           1960         2003
```

The results shows that the station of Phoenix Park, Markree, Roches Point Were built around 2005.So the three stations will be excluded in order to avoid massive N/A numbers.

```
stations_separate <- c(
"Markree",
"Phoenix Park",
"Roches Point"
)

weather_main <- weather_wide |>
filter(!Meteorological.Weather.Station %in% stations_separate)

#remaining 3 stations
weather_separated <- weather_wide |>
filter(Meteorological.Weather.Station %in% stations_separate)

head(weather_main)
```

```
# A tibble: 6 x 13
  Meteorological.Weather.Station Month mean_air_temperature mean_max_temperature
  <chr>                           <int>                <dbl>                <dbl>
1 Malin Head                          1                  5.7                  8
2 Claremorris                         1                  3.8                  7
3 Valentia Observatory                1                  6.6                  9.5
4 Belmullet                           1                  5.6                  8.3
5 Shannon Airport                     1                  4.7                  8.2
6 Dublin Airport                      1                  5.1                  7.7
# i 9 more variables: mean_min_temperature <dbl>, max_air_temperature <dbl>,
#   min_air_temperature <dbl>, grass_min_temperature <dbl>,
#   mean_wind_speed <dbl>, highest_gust <dbl>, sunshine_hours <dbl>,
#   precipitation_mm <dbl>, Year <int>
```

The variable of weather_main is the final data frame we are going to analyze.

1.2 analyze the data

1 Table of monthly temperature

With the preprocessed dataset weather_main, we can further analyze the weather data. The first part focuses on the temperature data.

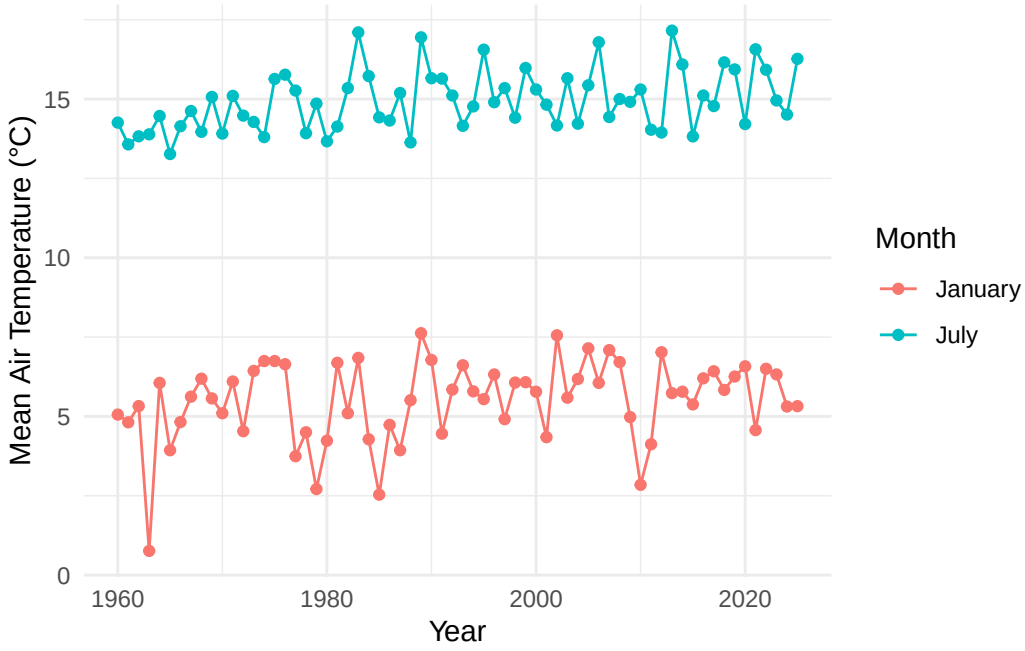
```
mean_temp_by_month <- weather_main |>
group_by(Month) |>
summarise(
mean_air_temperature = mean(mean_air_temperature, na.rm = TRUE),
.groups = "drop"
) |>
arrange(Month)

knitr::kable(
mean_temp_by_month,
col.names = c("Month", "Mean Air Temperature (°C)"),
digits = 2
)
```

Month	Mean Air Temperature (°C)
1	5.51
2	5.67
3	6.78
4	8.41
5	10.86
6	13.34
7	14.96
8	14.82
9	13.14
10	10.64
11	7.60
12	6.14

The above table shows the monthly mean air temperature in Ireland, illustrating the overall a seasonal climate pattern in Ireland.Generally speaking, Ireland has a temperate martime climate featured by cool winter and mild summer.More detailed result can be shown by a line plot.

2 Line plot of monthly temperature

```
#line plot

library(ggplot2)


temp_jan_jul <- weather_main |>
filter(Month %in% c(1, 7)) |>
group_by(Year, Month) |>
summarise(
mean_air_temperature = mean(mean_air_temperature, na.rm = TRUE),
.groups = "drop"
)


temp_jan_jul$Month <- factor(
temp_jan_jul$Month,
levels = c(1, 7),
labels = c("January", "July")
)


ggplot(temp_jan_jul, aes(x = Year, y = mean_air_temperature, color = Month)) +
geom_line() +
geom_point() +
labs(
x = "Year",
y = "Mean Air Temperature (°C)",
color = "Month"
) +
theme_minimal()
```



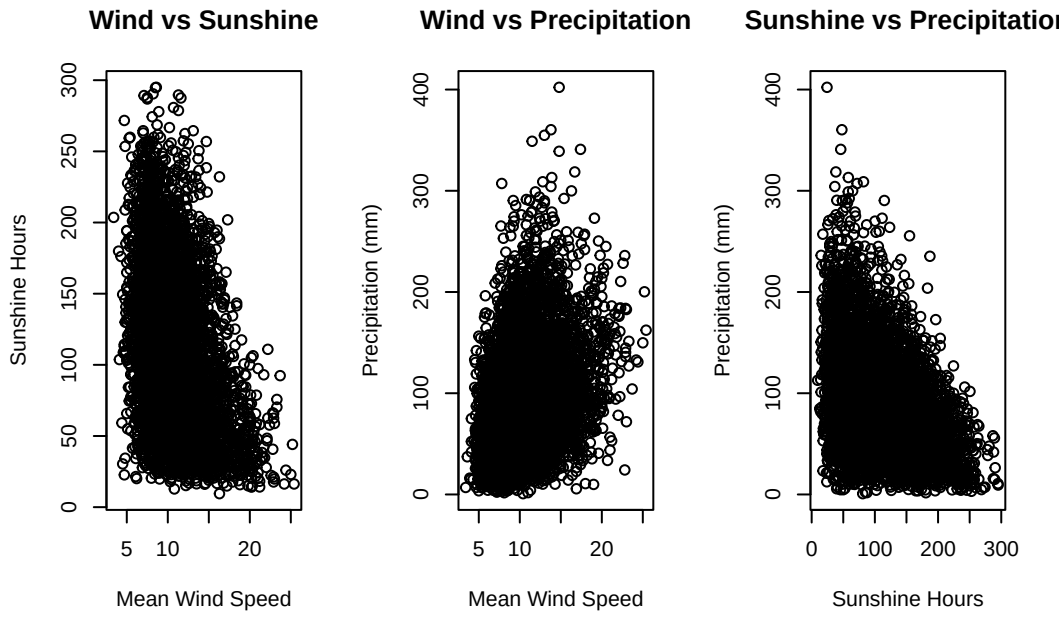
The above figure shows a slow but steady increasing trend in Ireland's temperature in both winter and summer. Over the past 60 years, the mean temperature in Ireland has increased by approximately 0.5 to 1 degree in both winter and summer. In addition, the temperature exhibits a periodic pattern with a cycle of approximately 4 to 5 years. During this period, years with lower temperatures are approximately 2.5 degrees cooler than years with higher temperatures.

Scatter plots of wind speed, sunshine hour and precipitation

Besides temperature, meteorological variables such as sunshine duration, wind, and precipitation are visualized using multiple plots.

```
par(mfrow = c(1, 3))

plot(weather_main$mean_wind_speed, weather_main$sunshine_hours,
     xlab = "Mean Wind Speed",
     ylab = "Sunshine Hours",
     main = "Wind vs Sunshine")

plot(weather_main$mean_wind_speed, weather_main$precipitation_mm,
     xlab = "Mean Wind Speed",
     ylab = "Precipitation (mm)",
```

```
     main = "Wind vs Precipitation")

plot(weather_main$sunshine_hours, weather_main$precipitation_mm,
     xlab = "Sunshine Hours",
     ylab = "Precipitation (mm)",
     main = "Sunshine vs Precipitation")
```



The three scatter plots indicate clear relationships among wind speed, sunshine hours, and precipitation. As wind speed and precipitation increase, sunshine hours tend to decrease, suggesting a negative relationship between sunshine duration and both wind speed and precipitation. In contrast, wind speed and precipitation exhibit a positive correlation.

part 2

2.1 Introduction and the purpose of the package

In this section, the **caret** package is introduced.

The package name stands for **Classification And REgression Training**, and it provides a unified interface for data preprocessing, model training, resampling, tuning, and evaluation. It is widely used in machine learning workflows in R.

Authors: Max Kuhn, et al. **CRAN link:** https://cran.r-project.org/package=caret

The following subsections demonstrate three core functionalities of the package using an original dataset (not from the package examples).

---

main functions in the package

2.2 Data preprocessing

First we need to install packages and load the dataset for demonstration. The Glass Identification Data Set contains 214 observations containing the type and the chmical properties of the glass.The goal is to predict correct glass type.

```
# install.packages("caret")
# install.packages("mlbench")
library(mlbench)
```

```
Warning: package 'mlbench' was built under R version 4.5.2
```

```
data(Glass)

library(caret)
```

```
Warning: package 'caret' was built under R version 4.5.2
```

```
Loading required package: lattice
```

```

Attaching package: 'caret'
```

```
The following object is masked from 'package:purrr':

    lift
```

In caret, the function used to preprocess data is preProcess(),the following code is used to show how to use the function.

```r
mini_df <- data.frame(
  a = c(1, 5, 10, 20),
  b = c(2, 4, 8, 16)
)
pp <- preProcess(mini_df, method = c("center", "scale"))
predict(pp, mini_df)
```

```
           a           b
1 -0.9749334 -0.88833014
2 -0.4874667 -0.56530100
3  0.1218667  0.08075729
4  1.3405334  1.37287385
```

The example show that the preprocess() function returns to an object, by using predict() function we can get the standardized data.Then similar procedure will be applied to the dataset.

```r
pp_model <- preProcess(
  Glass[, 1:9],
  method = c("center", "scale")
)

processed_glass <- predict(pp_model, Glass)
head(processed_glass)
```

```
          RI         Na        Mg         Al           Si          K         Ca
1  0.8708258  0.2842867 1.2517037 -0.6908222 -1.12444556 -0.67013422 -0.1454254
2 -0.2487502  0.5904328 0.6346799 -0.1700615  0.10207972 -0.02615193 -0.7918771
3 -0.7196308  0.1495824 0.6000157  0.1904651  0.43776033 -0.16414813 -0.8270103
4 -0.2322859 -0.2422846 0.6970756 -0.3102663 -0.05284979  0.11184428 -0.5178378
5 -0.3113148 -0.1688095 0.6485456 -0.4104126  0.55395746  0.08117845 -0.6232375
6 -0.7920739 -0.7566101 0.6416128  0.3506992  0.41193874  0.21917466 -0.6232375
          Ba         Fe Type
1 -0.3520514 -0.5850791    1
2 -0.3520514 -0.5850791    1
3 -0.3520514 -0.5850791    1
4 -0.3520514 -0.5850791    1
5 -0.3520514 -0.5850791    1
6 -0.3520514  2.0832652    1
```

```r
#divide the dataset into test dataset and train dataset
set.seed(123)
index <- createDataPartition(Glass$Type, p = 0.8, list = FALSE)

train_data <- Glass[index, ]
test_data  <- Glass[-index, ]

#preprocess the data

pp_model <- preProcess(train_data[,1:9], method = c("center","scale"))
train_processed <- predict(pp_model, train_data)
test_processed  <- predict(pp_model, test_data)
```

The code block above demonstrates how to preprocess the predictors of a dataset.In addition to the preprocess() function, The caret package also provides a simple function for splitting a dataset into a training set and a testing set.The function createDataPartition() returns random indices, and the parameter p controls the size of the training set. We can use the returned indices to create the training set and the test set.These indices can then be used to create the training set and the testing set.

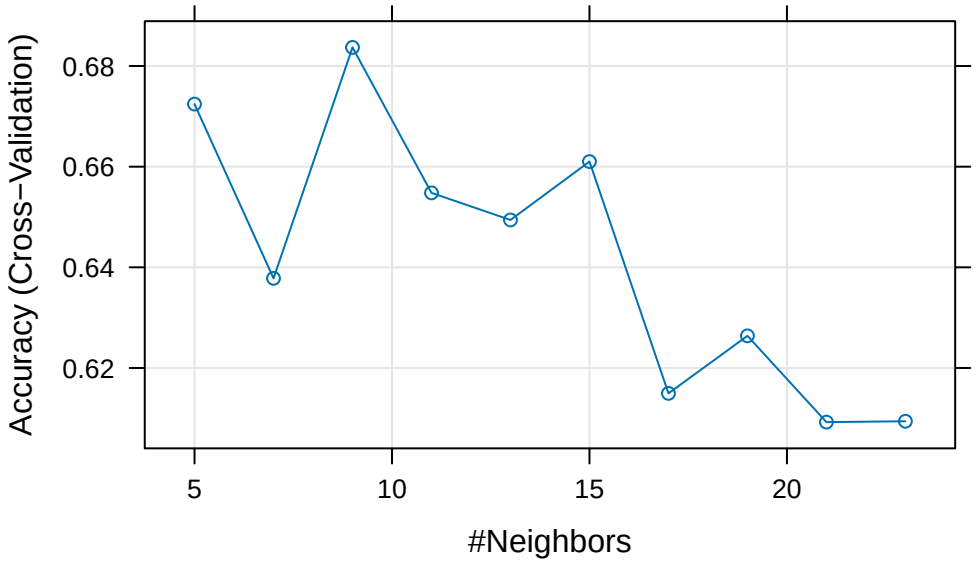
2.3 Model training using caret

```r
ctrl <- trainControl(method = "cv", number = 5)
knn_model <- train( Type ~ ., data = train_processed, method = "knn", trControl = ctrl, tunel
knn_model
```

```
k-Nearest Neighbors

174 samples
  9 predictor
  6 classes: '1', '2', '3', '5', '6', '7'

No pre-processing
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 139, 139, 140, 139, 139
Resampling results across tuning parameters:

  k   Accuracy   Kappa
   5  0.6724370  0.5407858
   7  0.6378151  0.4896468
```

```
  9  0.6836975  0.5555616
 11  0.6547899  0.5111599
 13  0.6494118  0.5034042
 15  0.6610084  0.5174333
 17  0.6149580  0.4486651
 19  0.6263866  0.4628964
 21  0.6092437  0.4364717
 23  0.6094118  0.4386431

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was k = 9.
```

```
plot(knn_model)
```



During the model training, trainControl() function offers us automatic n-fold cross validation. Then the train() function receives parameters such as the method = "knn", tuneLength = 10 which choose the ML model and model parameters. This plot shows the cross-validation accuracy for different k values. The accuracy peaks when k is around 9 and decreases when k becomes too small or too large.We can see the best accuracy it can reach is about 0.68

2.4 Model evaluation and results

```
pred <- predict(knn_model, test_processed)
confusionMatrix(pred, test_processed$Type)
```

```
Confusion Matrix and Statistics

          Reference
Prediction  1  2  3  5  6  7
         1 10  5  2  0  0  0
         2  4  9  1  1  0  1
         3  0  0  0  0  0  0
         5  0  1  0  0  0  0
         6  0  0  0  0  1  0
         7  0  0  0  1  0  4

Overall Statistics

               Accuracy : 0.6
                 95% CI : (0.4333, 0.7514)
    No Information Rate : 0.375
    P-Value [Acc > NIR] : 0.003206

                  Kappa : 0.415

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: 1 Class: 2 Class: 3 Class: 5 Class: 6 Class: 7
Sensitivity            0.7143   0.6000    0.000   0.0000    1.000   0.8000
Specificity            0.7308   0.7200    1.000   0.9737    1.000   0.9714
Pos Pred Value         0.5882   0.5625      NaN   0.0000    1.000   0.8000
Neg Pred Value         0.8261   0.7500    0.925   0.9487    1.000   0.9714
Prevalence             0.3500   0.3750    0.075   0.0500    0.025   0.1250
Detection Rate         0.2500   0.2250    0.000   0.0000    0.025   0.1000
Detection Prevalence   0.4250   0.4000    0.000   0.0250    0.025   0.1250
Balanced Accuracy      0.7225   0.6600    0.500   0.4868    1.000   0.8857
```

Finally, the predict() function is used to apply the KNN model.It produces the predicted class for each sample in the test set.And by confusionMatrix function we can finnally evaluate our model on the test dataset.

Part 3 Functions/Programming

In this section, a function is implemented to perform a one-dimensional Metropolis–Hastings (MH) algorithm within the Markov Chain Monte Carlo (MCMC) framework.

MCMC is a class of algorithms used to generate samples from complex probability distributions that are difficult to sample from directly.In the algorithm, we need to specify the goal distribution, the initial guess value and a proposal distribution to establish the MCMC chains.In each iteration, the algorithm will propose a new candidate acoording to our proposal distribution and decide whether to accept the new candidate according to the value of target distribution's density function.

In general MCMC chains, the proposal can be chosen as any distributions.However, in the implementation only Normal distribution is used to simplify the algorithm.

3.1 Define the function

```
############################################################
# my_mcmc(): one dimensional Metropolis-Hastings MCMC function
#
#  parameters:

#   target      - goal distribution, both input and output is a numerical value
#   init        - initial values
#   n_iter      - iterations
#   proposal_sd - the standard deviation of Normal proposal
#
#   Return
#   An S3 list object with class my_mcmc including:
#     samples
#     accept_rate
#     init
#     proposal_sd
#     n_iter
#
#
#   - target can be any one dimensional distribution
############################################################

my_mcmc <- function(target, init, n_iter = 10000, proposal_sd = 1) {

  if (!is.function(target)) {
```

```
  stop("'target' must be a function.")
}
if (!is.numeric(init) || length(init) != 1) {
  stop("'init' must be a numeric scalar (length 1).")
}
if (!is.numeric(n_iter) || n_iter <= 0) {
  stop("'n_iter' must be a positive integer.")
}
if (!is.numeric(proposal_sd) || proposal_sd <= 0) {
  stop("'proposal_sd' must be a positive number.")
}

samples <- numeric(n_iter)
samples[1] <- init
current <- init
current_target <- target(current)

if (!is.finite(current_target) || current_target < 0) {
  stop("initial desitity is not a positive number")
}

n_accept <- 0

for (i in 2:n_iter) {
  proposal <- rnorm(1, mean = current, sd = proposal_sd)
  proposal_target <- target(proposal)

  if (!is.finite(proposal_target) || proposal_target < 0) {
    proposal_target <- 0
  }

  # calculate accept rate  alpha = min(1, target(proposal)/target(current))
  if (current_target == 0 && proposal_target == 0) {
    alpha <- 0
  } else if (current_target == 0 && proposal_target > 0) {
    alpha <- 1
  } else {
    alpha <- min(1, proposal_target / current_target)
  }

  # accept or reject
  if (runif(1) < alpha) {
```

```
      current <- proposal
      current_target <- proposal_target
      n_accept <- n_accept + 1
    }
    samples[i] <- current
  }

  # overall accept rate
  accept_rate <- n_accept / (n_iter - 1)

  #S3 object
  res <- list(
    samples     = samples,
    accept_rate = accept_rate,
    init        = init,
    proposal_sd = proposal_sd,
    n_iter      = n_iter
  )
  class(res) <- "my_mcmc"
  return(res)
}
```

In the implementation, before the MCMC procedure begins, the function first performs input validation. It then executes the MCMC algorithm.Finally, it returns an S3 object containing the MCMC samples and relevant diagnostic information.

```
############################################################
# print.my_mcmc()
############################################################

print.my_mcmc <- function(x, ...) {
  cat("MCMC object of class 'my_mcmc'\n")
  cat("------------------------------------------------\n")
  cat("  Iterations      :", x$n_iter, "\n")
  cat("  Initial value   :", x$init, "\n")
  cat("  Proposal SD     :", x$proposal_sd, "\n")
  cat("  Acceptance rate :", round(x$accept_rate, 3), "\n")
  cat("------------------------------------------------\n")
  cat("  First 5 samples : ",
      paste(round(head(x$samples, 5), 3), collapse = ", "), "\n")
  invisible(x)
}
```

The print() function only provides us with some basic information and the first 5 samples.

3.2 print(),summary() and plot()

```r
################################################################
# summary.my_mcmc():
################################################################

summary.my_mcmc <- function(object, ...) {
  s <- object$samples
  n <- length(s)

  stats <- list()
  stats$n_iter       <- object$n_iter
  stats$accept_rate  <- object$accept_rate
  stats$mean         <- mean(s)
  stats$sd           <- sd(s)
  stats$quantiles    <- quantile(s, probs = c(0.025, 0.5, 0.975))


  class(stats) <- "summary.my_mcmc"
  return(stats)
}

################################################################
# print.summary.my_mcmc()
################################################################

print.summary.my_mcmc <- function(x, ...) {
  cat("Summary of 'my_mcmc' object\n")
  cat("-------------------------------------------------\n")
  cat("Iterations      :", x$n_iter, "\n")
  cat("Acceptance rate :", round(x$accept_rate, 3), "\n")
  cat("-------------------------------------------------\n")
  cat("Posterior sample statistics:\n")
  cat("  Mean           :", round(x$mean, 3), "\n")
  cat("  SD             :", round(x$sd, 3), "\n")
  cat("  2.5% quantile  :", round(x$quantiles[1], 3), "\n")
  cat("  Median         :", round(x$quantiles[2], 3), "\n")
  cat("  97.5% quantile :", round(x$quantiles[3], 3), "\n")
```

```
  invisible(x)
}
```

Unlike the print() function, the summary() function provides additional statistics, such as quantiles, the mean, and the standard deviation..

```
#############################################################
# plot.my_mcmc(): method to plot
# 2 plots will be drawn by default
#   1) trace plot
#   2) density plot
#############################################################

plot.my_mcmc <- function(x, ...) {
  s <- x$samples

  old_par <- par(no.readonly = TRUE)
  on.exit(par(old_par))

  par(mfrow = c(1, 2))

  # 1. Trace plot
  plot(s, type = "l",
       main = "Trace plot of MCMC samples",
       xlab = "Iteration",
       ylab = "Value")
  # 2. density plot
  dens <- density(s)
  plot(dens,
       main = "Kernel density estimate of samples",
       xlab = "Value")


}
```

The plot() function performs 2 plots, the trace of MCMC and the density of our Monte Carlo samples.The first is used to judge the convergence and the second is used to show the kernel density of the Monte Carlo result.

3.3 example of the function

```
############################################################
# Example: MCMC Sampling from a Bimodal Target Distribution
#
# goal distribution
#   p(x) = 0.3 N(-3, 1^2) + 0.7 N(3, 1^2)
#
############################################################


target_bimodal <- function(x) {
  0.3 * dnorm(x, mean = -3, sd = 1) +
    0.7 * dnorm(x, mean = 3, sd = 1)
}

#   MCMC
set.seed(123)
mcmc_fit <- my_mcmc(
  target      = target_bimodal,
  init        = 0,
  n_iter      = 8000,
  proposal_sd = 1
)


print(mcmc_fit)
```

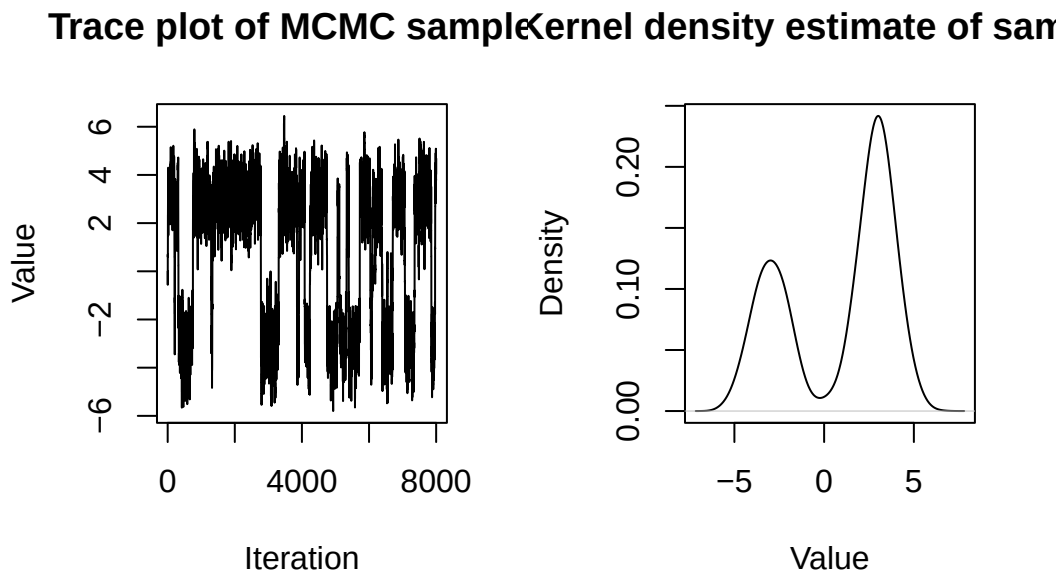
```
MCMC object of class 'my_mcmc'
------------------------------------------------
  Iterations      : 8000
  Initial value   : 0
  Proposal SD     : 1
  Acceptance rate : 0.716
------------------------------------------------
  First 5 samples :  0, -0.56, 0.63, 0.7, 0.591
```

```
summary(mcmc_fit)
```

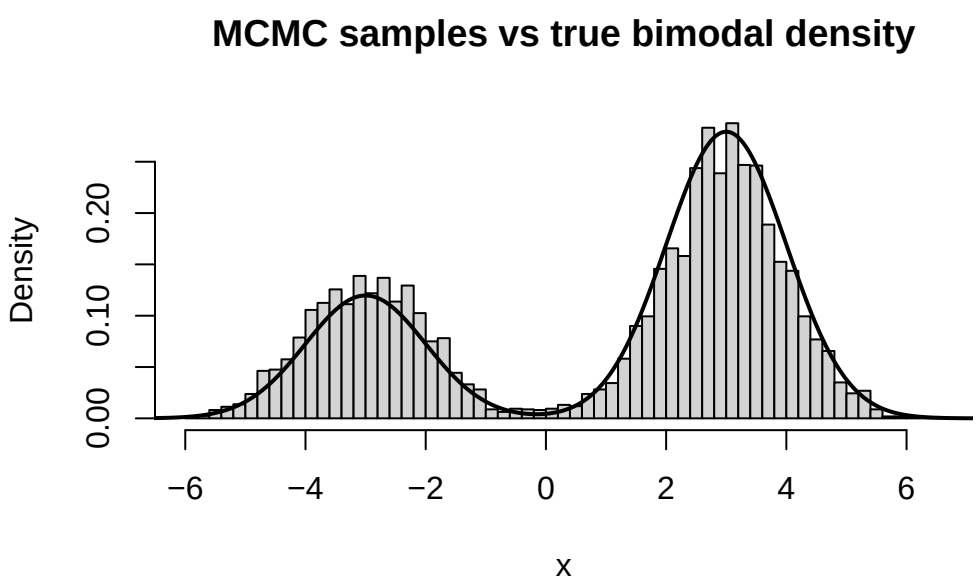
```
Summary of 'my_mcmc' object
```

```
------------------------------------------------
Iterations      : 8000
Acceptance rate : 0.716
------------------------------------------------
Posterior sample statistics:
  Mean            : 0.848
  SD              : 3.028
  2.5% quantile   : -4.513
  Median          : 2.253
  97.5% quantile : 4.704
```

```r
plot(mcmc_fit)
```




```r
# the true density
x_grid <- seq(-8, 8, length.out = 400)
true_density <- 0.3 * dnorm(x_grid, -3, 1) + 0.7 * dnorm(x_grid, 3, 1)

#contranstion between
hist(mcmc_fit$samples, breaks = 50, freq = FALSE,
     main = "MCMC samples vs true bimodal density",
     xlab = "x")
lines(x_grid, true_density, lwd = 2)
```



The example is the MCmc of a bimodal distribution $p(x) = 0.3N(-3, 1^2) + 0.7N(3, 1^2)$. From the result we can see the function successfully simulate the samples from the target distribution.